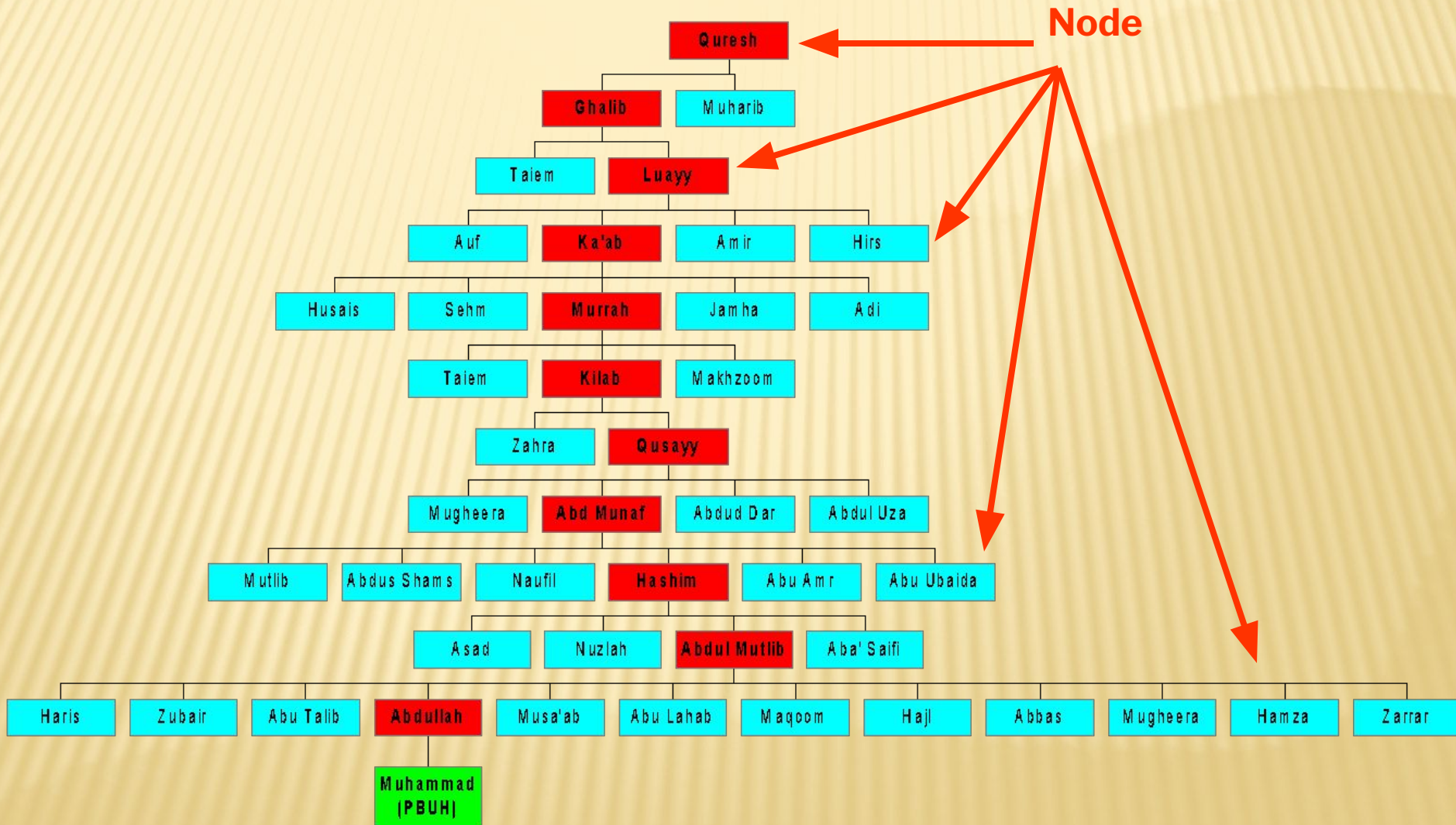


TREES

TREES



FAMILY TREE



TREE - DEFINITION

A tree is a finite set of one or more nodes such that:

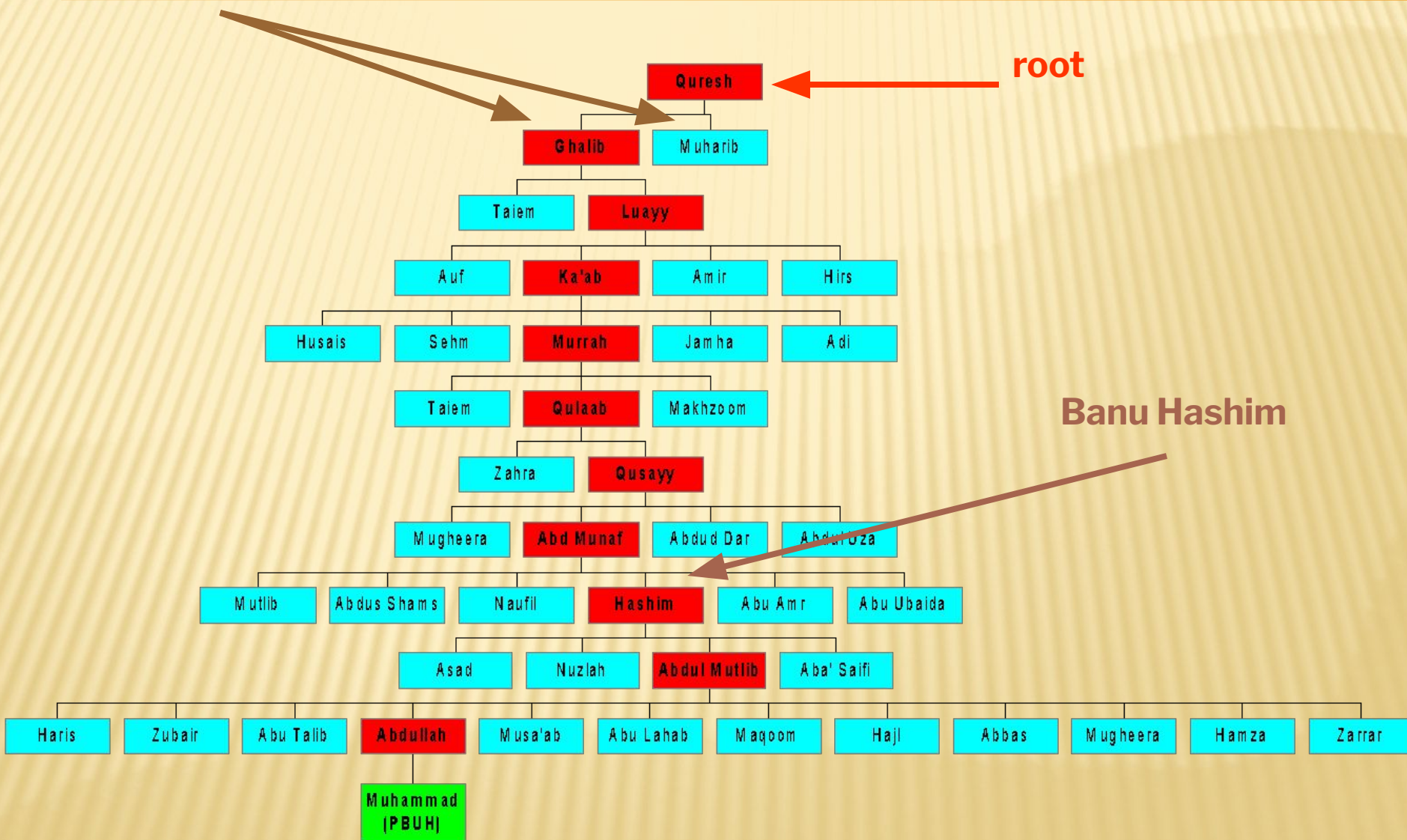
1. There is a specially designated node called the *root*.
2. The remaining nodes are partitioned in $n \geq 0$ disjoint sets $T_1, T_2, \dots, T_n$, where each of these sets is a tree.
3. $T_1, T_2, \dots, T_n$ are called the *sub-trees* of the root.

Recursive Definition

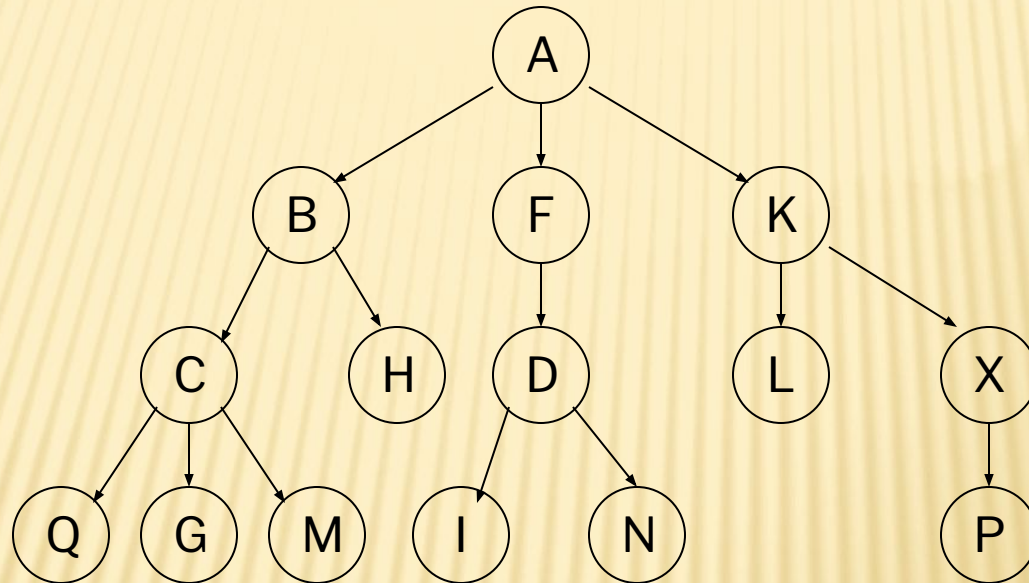


FAMILY TREE

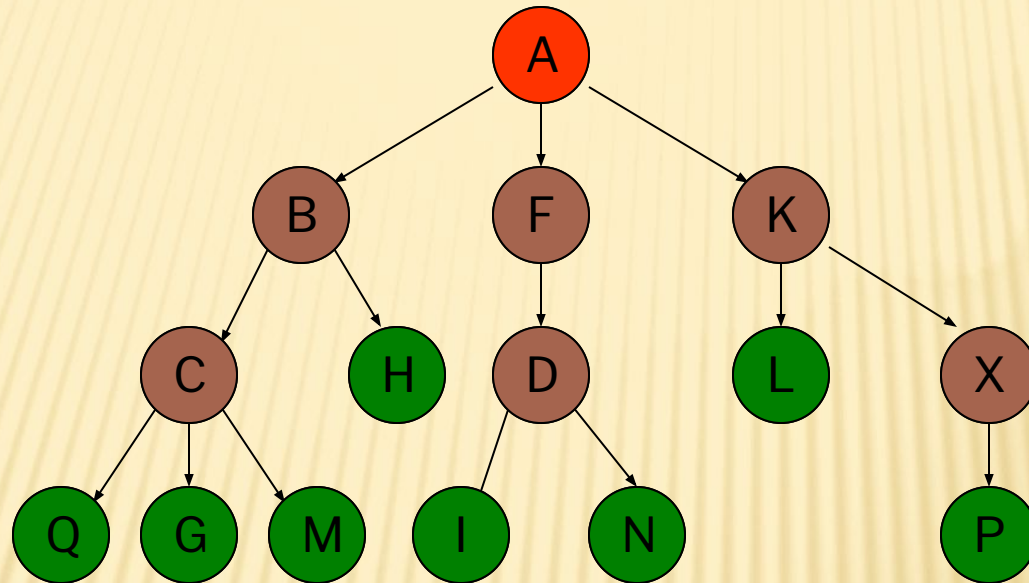
Sub-trees



TREE



NODE TYPES



Root (no parent)



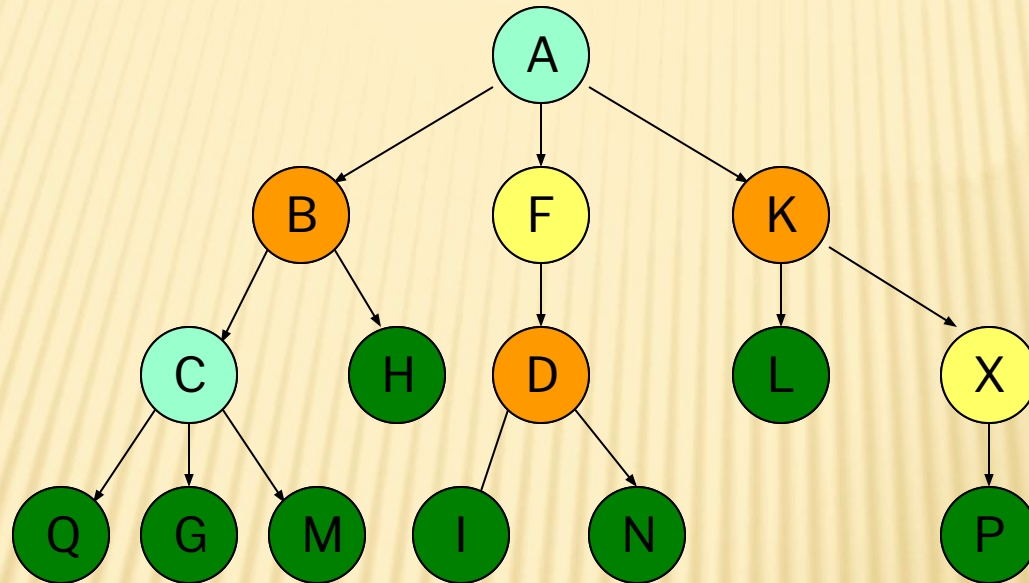
Intermediate nodes (has a parent and at least one child)



Leaf nodes (0 children)

DEGREE OF A NODE

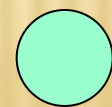
NUMBER OF CHILDREN



Degree 1



Degree 0

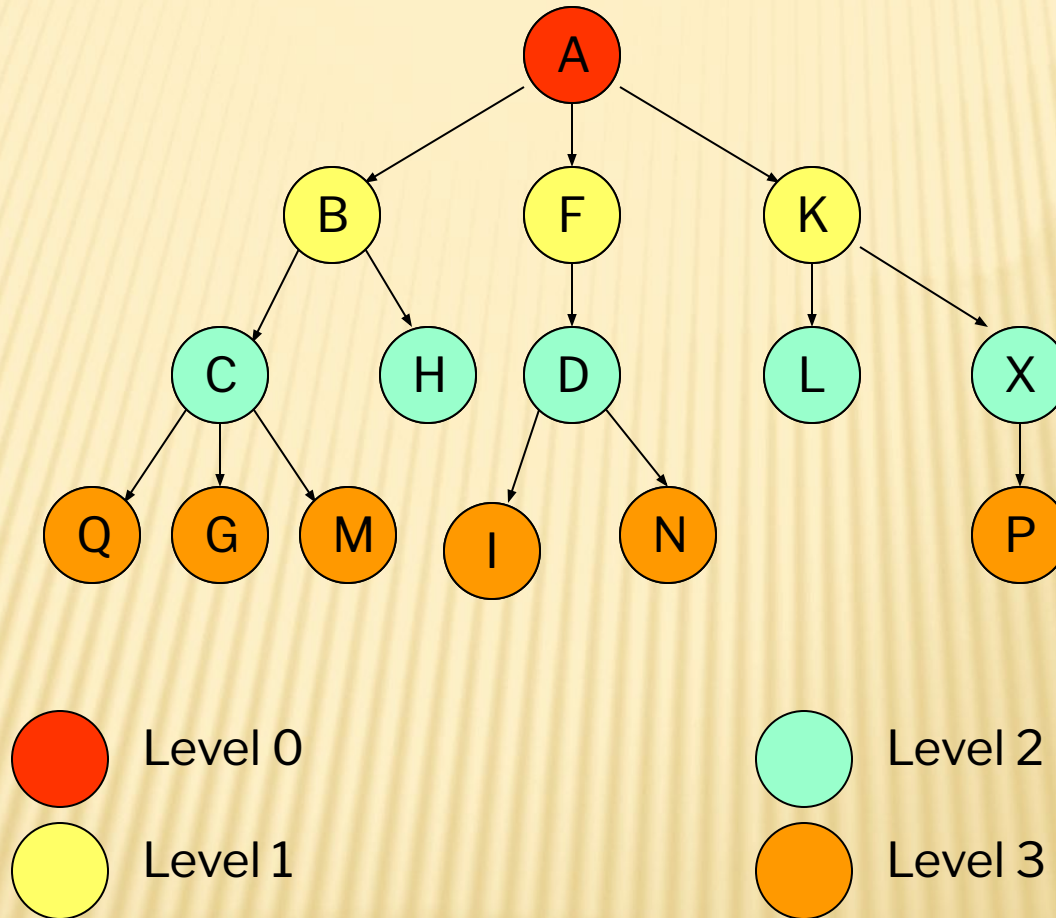


Degree 3



Degree 2

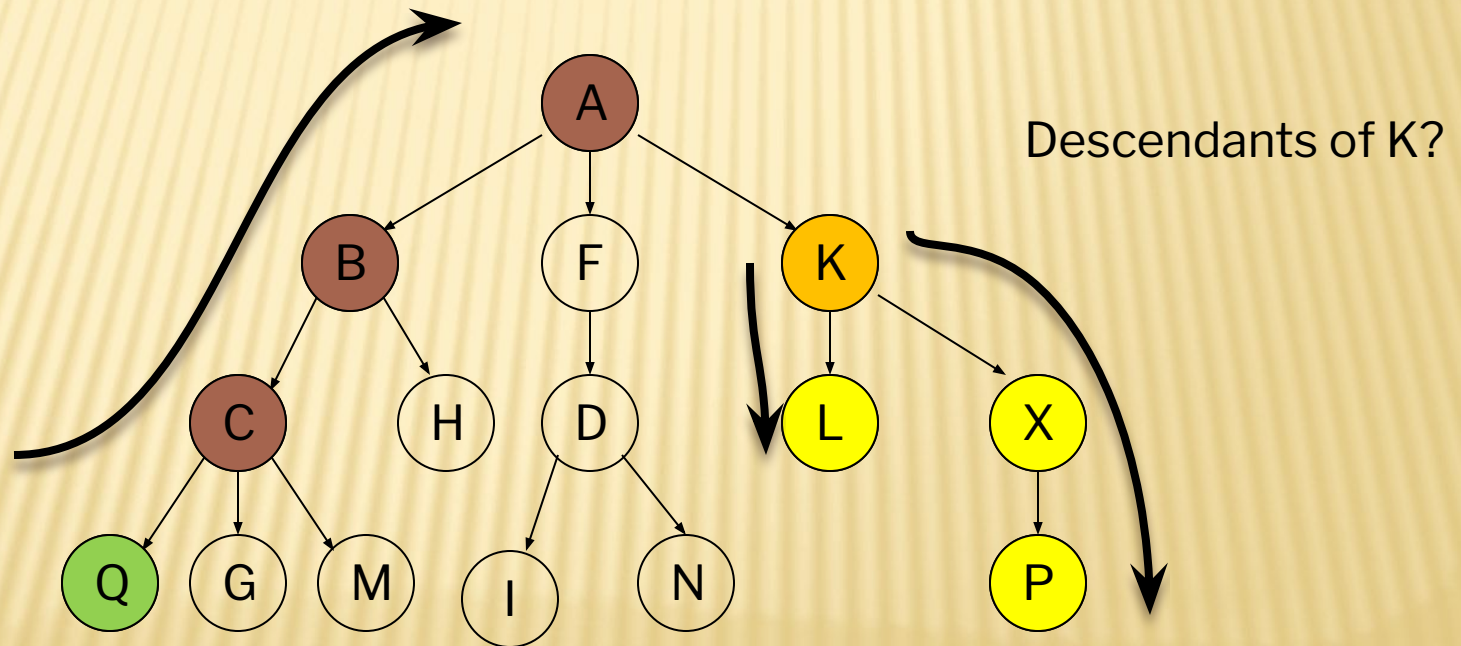
LEVEL OF A NODE DISTANCE FROM THE ROOT



Height of the Tree = Maximum Level + 1

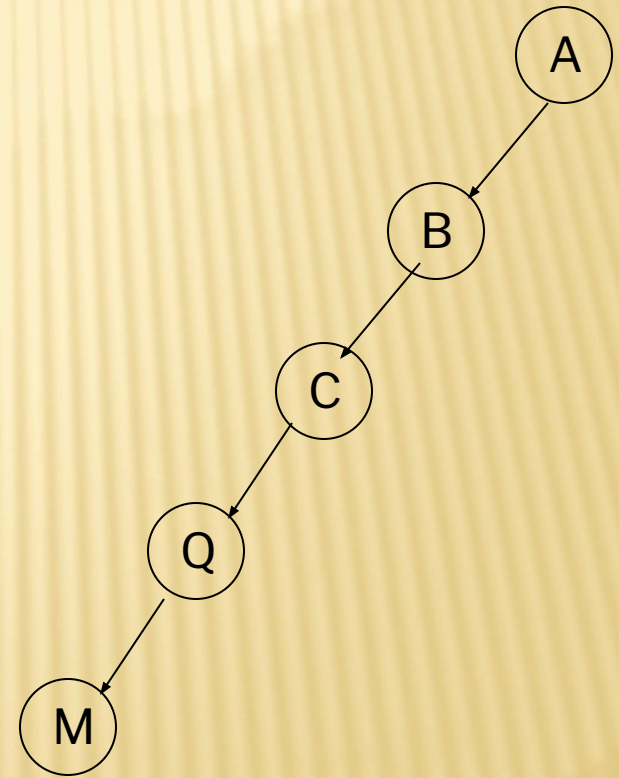
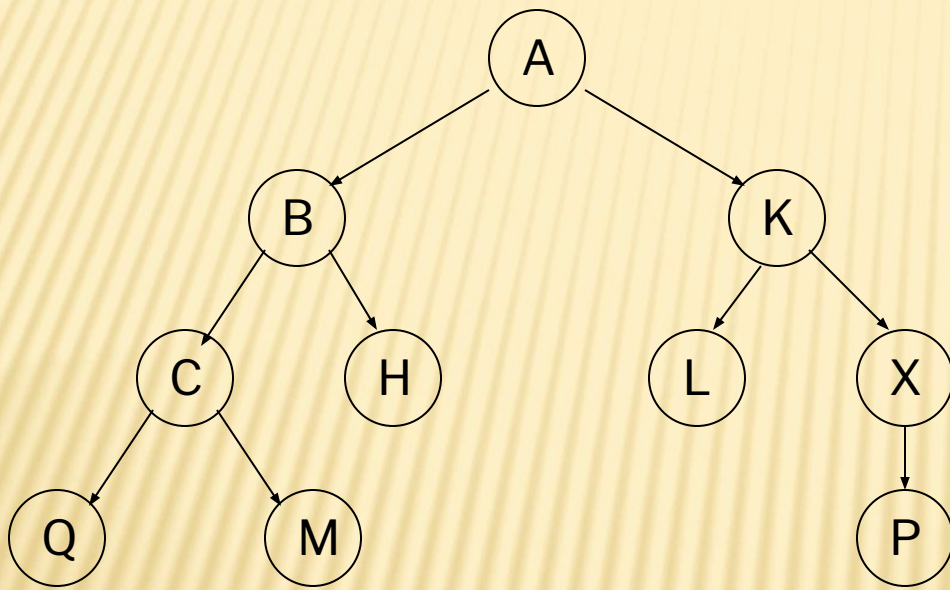
ANCESTORS

- Ancestors of a node: parent, grandparent, grand-grandparent, etc.
- Descendant of a node: child, grandchild, grand-grandchild, etc.



Ancestors of Q?

EXAMPLES OF BINARY TREES

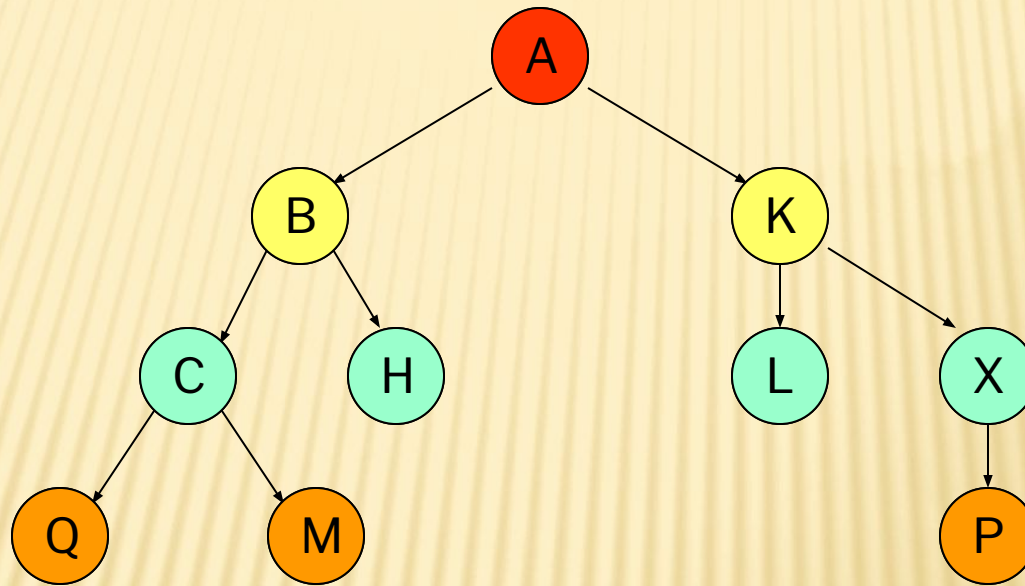


PROPERTIES OF BINARY TREES

- The maximum number of nodes on level i of a binary tree is 2^i
- The maximum number of nodes in a binary tree of height k is $2^k - 1$

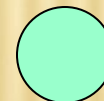
MAXIMUM NUMBER OF NODES ON A LEVEL

- The maximum number of nodes on level i of a binary tree is 2^i



 Level 0 $2^0 = 1$

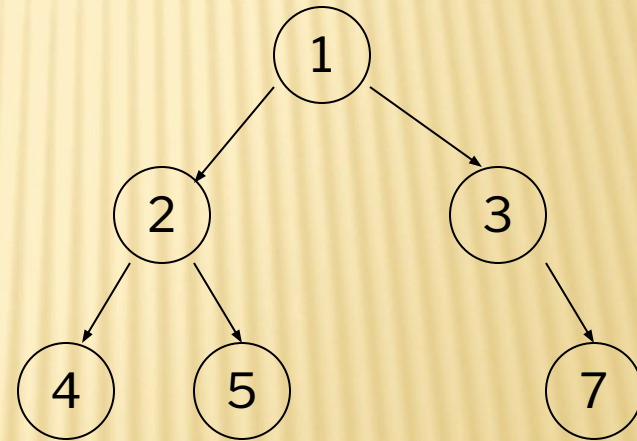
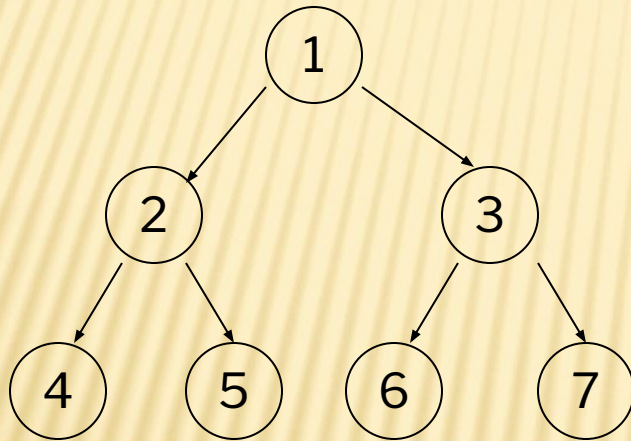
 Level 1 $2^1 = 2$

 Level 2 $2^2 = 4$

 Level 3 $2^3 = 8$

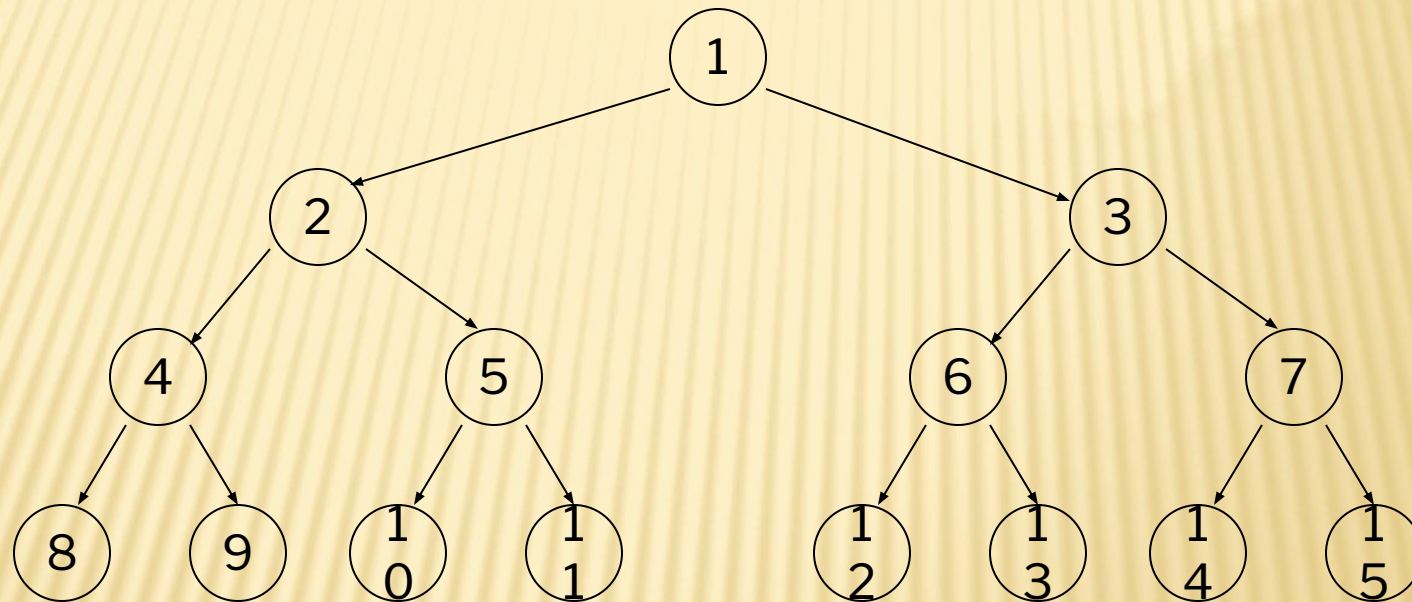
MAXIMUM NUMBER OF NODES IN A BINARY TREE

- The maximum number of nodes in a binary tree of height k is $2^k - 1$



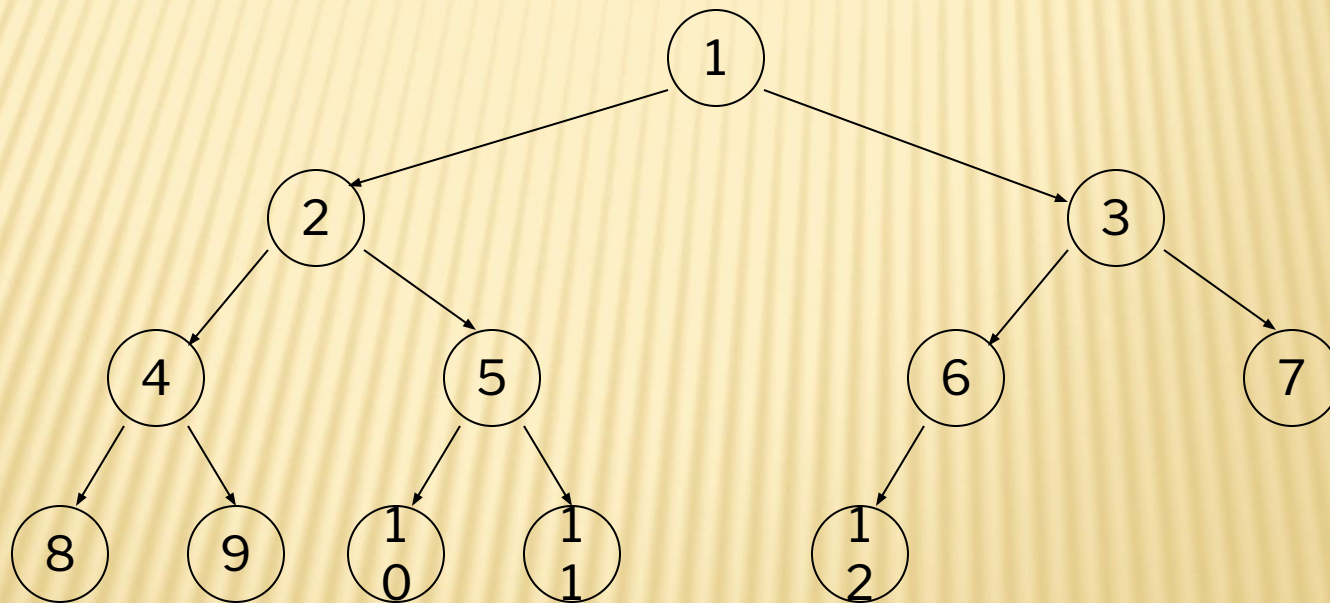
FULL BINARY TREE

A binary tree of height k having $2^k - 1$ nodes is called a **full** binary tree



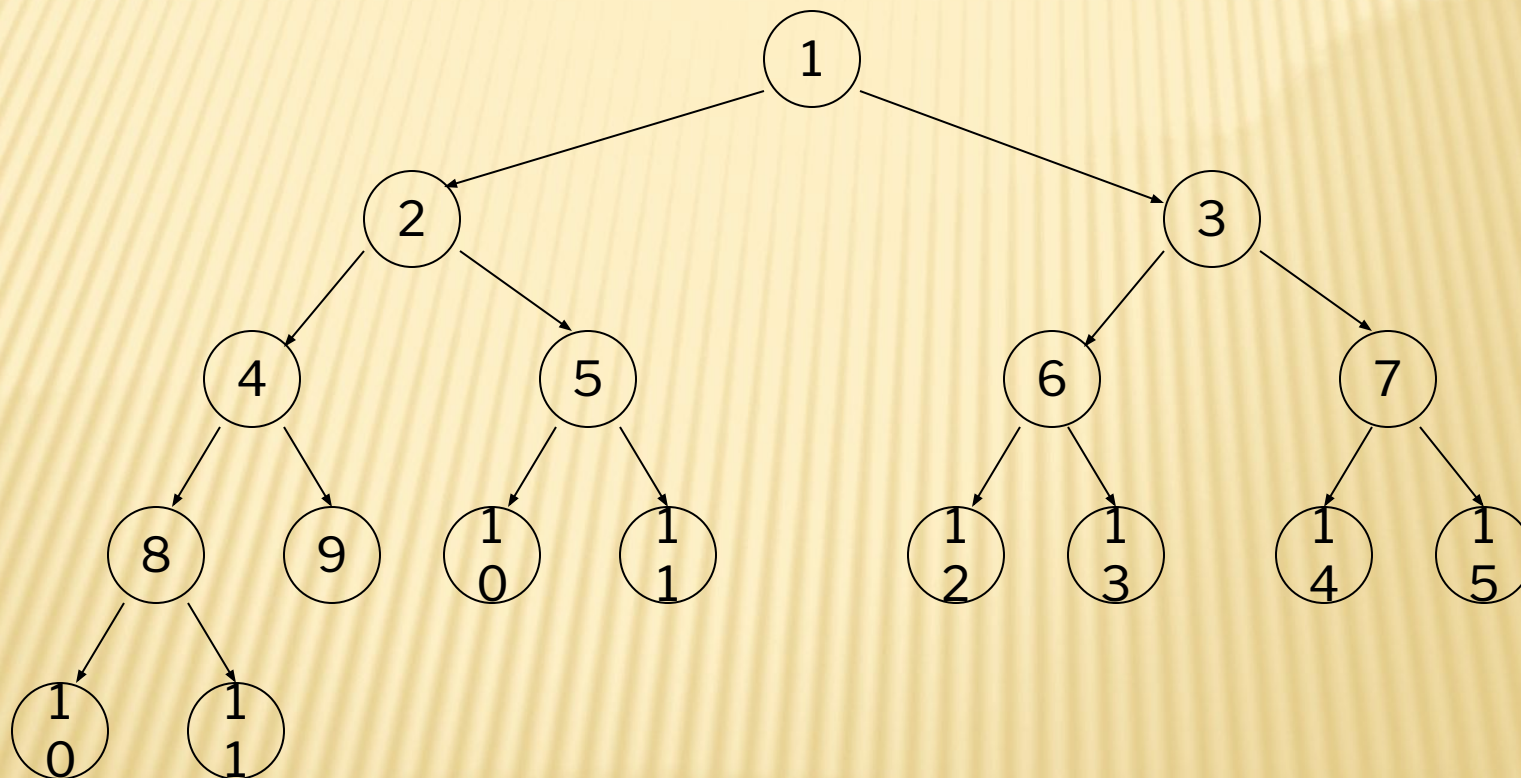
COMPLETE BINARY TREE

A binary tree that is completely filled, with the possible exception of the bottom level, which is filled from left to right, is called a **complete** binary tree



COMPLETE BINARY TREE

Is it a complete binary tree?



BINARY TREE

NO NODE HAS A DEGREE > 2

```
struct TreeNode {  
    int      data;  
    TreeNode *left, *right; // left subtree and right subtree  
};
```

```
Class BinaryTree {  
    private:  
        TreeNode * root;  
    public:  
        BinaryTree() { root = NULL; }  
        void add (int data);  
        void remove (int data);  
        void InOrder();      // In order traversal  
        ~ BinaryTree();  
};
```

BINARY TREE TRAVERSAL

IN ORDER TRAVERSAL (LNR)

```
void BinaryTree::InOrder()    // work horse function
{
    InOrder(root);
}
```

```
void BinaryTree::InOrder(TreeNode *t)
{
    if (t) {
        InOrder(t->left);
        visit(t);
        InOrder(t->right);
    }
}
```

```
void BinaryTree::visit (TreeNode *t) { cout << t->data; }
```

BINARY TREE TRAVERSAL

IN ORDER TRAVERSAL (LNR)

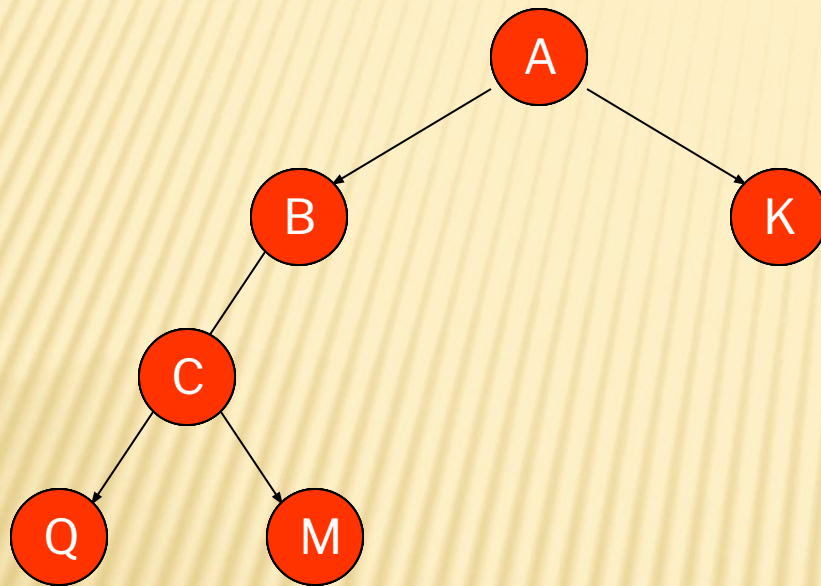
```
if (t) {
```

```
    InOrder(t->left);
```

```
    visit(t);
```

```
    InOrder(t->right);
```

```
}
```



InOrder(NULL)

InOrder(Q)
)

InOrder(NULL)

InOrder(B)

InOrder(A)

InOrder(M->left)

visit(M)

InOrder(M->right)

t=M

InOrder(C->left)

visit(C)

InOrder(C->right)

t=C

InOrder(K->left)

visit(K)

InOrder(K->right)

t=K

InOrder(A->left)

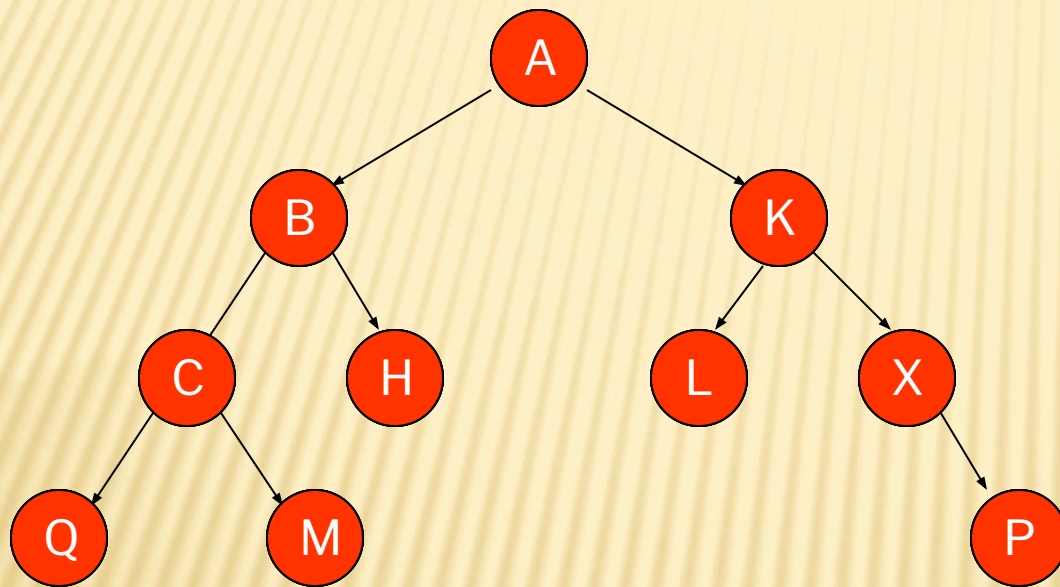
visit(A)

InOrder(A->right)

t=A

BINARY TREE TRAVERSAL

IN ORDER TRAVERSAL (LNR)



Q C M B H A L K X P

```
if (t) {
```

```
    InOrder(t->left);
```

```
    visit(t);
```

```
    InOrder(t->right);
```

```
}
```

BINARY TREE TRAVERSAL

PRE ORDER TRAVERSAL (NLR)

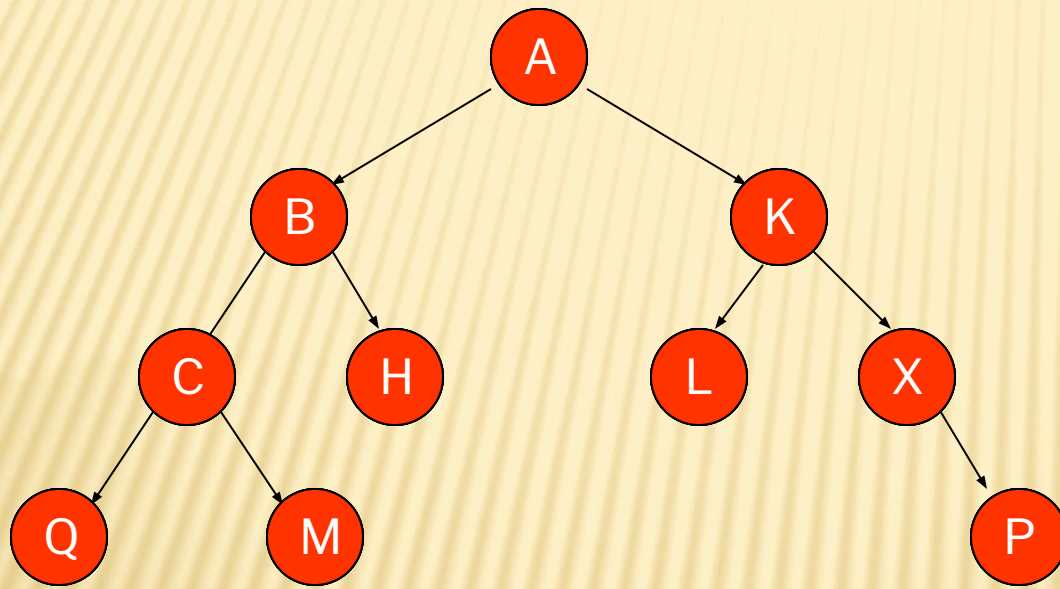
```
void BinaryTree::PreOrder()
{
    PreOrder(root);
}
```

```
void BinaryTree::PreOrder(TreeNode *t)
{
    if (t) {
        visit(t);
        PreOrder(t->left);
        PreOrder(t->right);
    }
}
```

```
void BinaryTree::visit (TreeNode *t) { cout << t->data; }
```

BINARY TREE TRAVERSAL

PRE ORDER TRAVERSAL (NLR)



A B C Q M H K L X P

```
if (t) {
```

```
    visit(t);
```

```
    PreOrder(t->left);
```

```
    PreOrder(t->right);
```

```
}
```


BINARY TREE TRAVERSAL

POST ORDER TRAVERSAL (LRN)

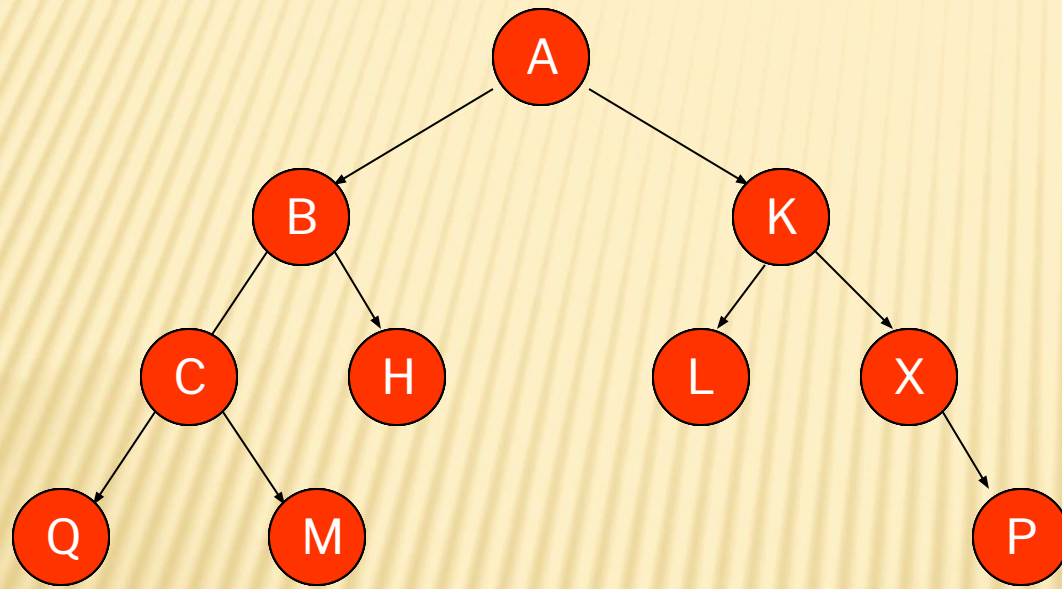
```
void BinaryTree::PostOrder()    // work horse function
{
    PostOrder(root);
}
```

```
void BinaryTree::PostOrder(TreeNode *t)
{
    if (t) {
        PostOrder(t->left);
        PostOrder(t->right);
        visit(t);
    }
}
```

```
void BinaryTree::visit (TreeNode *t) { cout << t->data; }
```

BINARY TREE TRAVERSAL

POST ORDER TRAVERSAL (LRN)



```
if (t) {
```

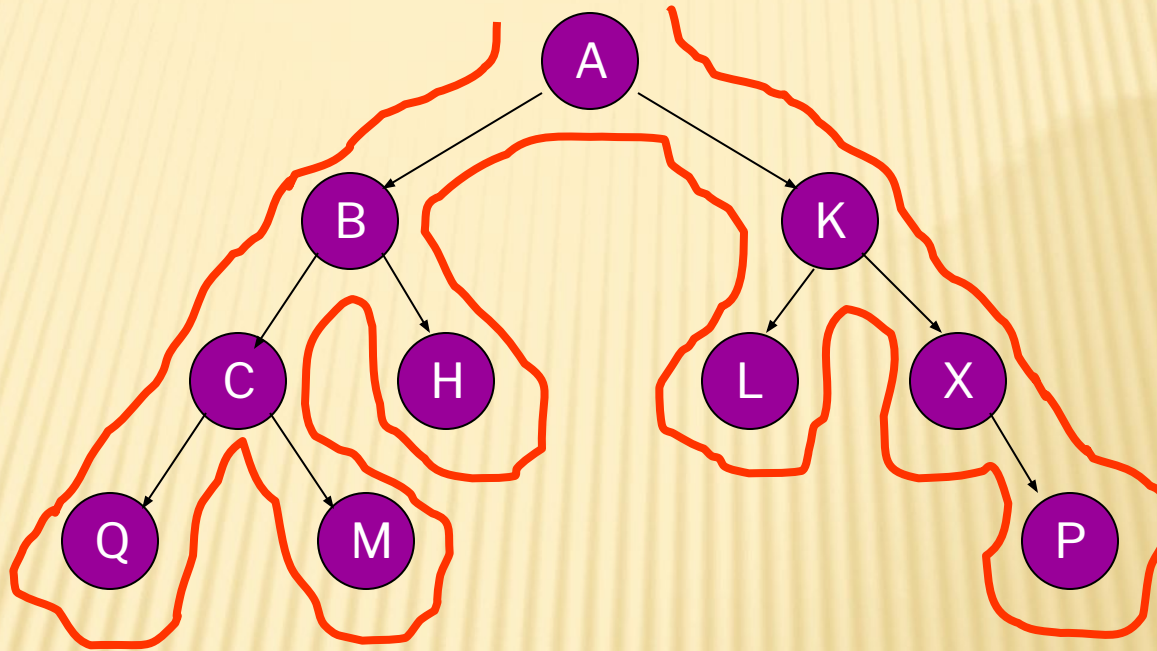
```
    PostOrder(t->left);
```

```
    PostOrder(t->right);
```

```
    visit(t);
```

```
}
```

BINARY TREE TRAVERSAL



NLR – visit when at the left of the Node

A B C Q M H K L X P

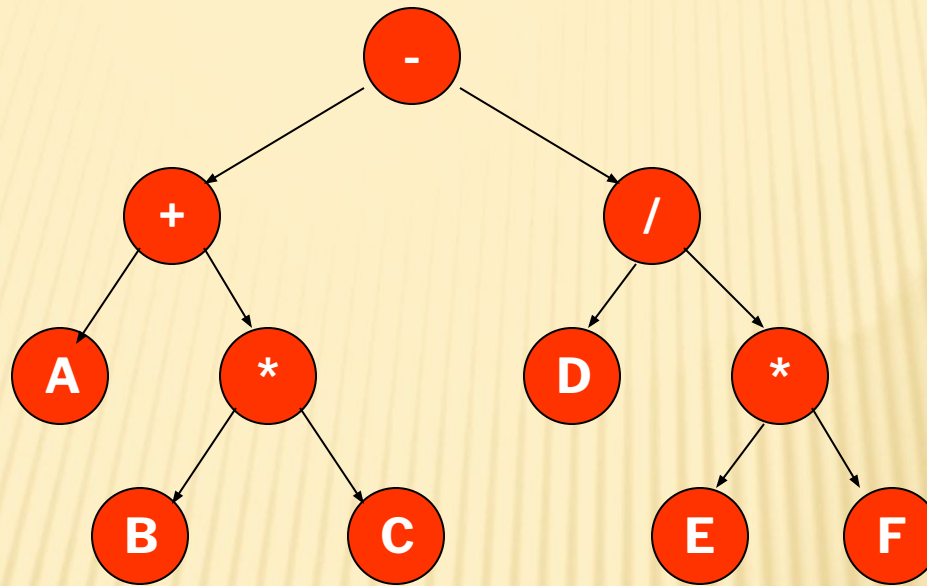
LNR – visit when under the Node

Q C M B H A L K X P

LRN – visit when at the right of the Node

Q M C H B L P X K A

Expression Tree



LNR: A+B*C-D/E*F

NLR: -+A*BC/D*EF

LRN: ABC*+DEF*/-

BINARYTREE ::~ BINARYTREE();

Which Algorithm?

Delete both the left child and right child
before deleting itself

LRN